Minor Project – III (MCA-269) — Synopsis

Programme: Master of Computer Applications, Semester III · Batch 2025–2027
Institution: Jagan Institute of Management Studies, Rohini **Submitted by:** Ashwani Kumar Puri *(Roll No. — to be filled)* **Guide:** *(to be filled)* **Date:** 7 September 2026

Prepared to the synopsis format prescribed in the MCA-269 Guidelines (Batch 2025–2027), §"Format of the Student Project Report", items 1–7.

---

1. Title of Project

> **AI Tutor — An Evidence-Based Adaptive Learning System with Deliberate Scaffolding Withdrawal**

Product positioning in one line: *an AI tutor whose objective is to make itself unnecessary.*

---

2. Problems with the Existing System

The problem addressed here is not hypothetical. It was observed directly, over a six-week period, by the author while learning a modern web development stack under time pressure. The failure modes below are documented from that experience and are reproducible across the wider population of self-directed technical learners.

2.1 The existing workflow

How a self-directed learner acquires a technical skill today:

#	Step	Mechanism	Artefact produced
1	Learner identifies a skill to acquire	Job posting, curiosity, curriculum	—

#	Step	Mechanism	Artefact produced
2	Learner selects a course	YouTube, Udemy, Coursera, documentation	Course URL
3	Learner watches content sequentially	Video player, linear playlist	Watch history
4	Learner follows along, typing what is shown	Editor alongside video	Copy of instructor's code
5	Learner marks lessons complete	Platform checkbox	Completion percentage
6	Learner attempts independent work	Blank editor	Frequently nothing
7	Learner becomes stuck	—	—
8	Learner asks an AI assistant	ChatGPT, Copilot, Claude	Working code, supplied
9	Learner proceeds, problem resolved	—	No retained capability
10	Learner returns weeks later	—	Material substantially forgotten

2.2 Identified deficiencies

1. **Completion is measured instead of capability.** Every platform records what has been watched or ticked. None records what the learner can independently produce. A 100% completion figure is compatible with zero ability to write the code unaided.
2. **Recognition is trained; recall is not.** Watching an instructor write code is a recognition task. Producing code from a specification is a recall and construction task. These are distinct cognitive operations and competence in the first does not transfer to the second. The learner experiences the feeling of understanding throughout, and discovers its absence only when asked to build something.
3. **Curricula are linear; learners are not.** A course is a fixed sequence authored for a median learner who does not exist. Time spent on already-known material is

time removed from unknown material. The author's own diagnostic scored 10/11 on reading code and 0/4 on writing it — a profile no linear course can respond to.

4. **No system maintains a model of what the learner knows.** Absent a representation of current state, no defensible next step can be selected. The learner selects the next video, which is not the same as selecting the next thing to learn.
5. **Generative AI has made the problem materially worse.** An AI assistant asked for an explanation supplies one; asked for code, supplies code. The learner's productivity rises and their capability does not, because effortful retrieval — the struggle — is the mechanism by which memory is formed. Existing AI tutoring products optimise for perceived helpfulness in each individual interaction, which is precisely the wrong objective function for learning.
6. **Nothing survives disuse.** Material learned in week one, never revisited, is substantially lost by week five. No mainstream product schedules revision against a model of individual forgetting.
7. **The blank page terminates sessions.** For learners with attention-regulation difficulties in particular, an unstructured task with no obvious first action does not produce slow progress — it produces abandonment. Session abandonment is the only true failure mode, and no existing product designs against it.

2.3 Cost of the existing system

For the individual learner, the cost is measured in months. Six weeks of full-time study producing recognition without production capability is six weeks that must be substantially repeated. Where the learner is preparing for employment, this delay carries direct financial cost in deferred salary.

At market scale, the cost is visible in completion statistics. Massive open online courses are widely reported to exhibit single-digit completion rates, and the proportion of *completers* who can subsequently work unaided is not measured by any provider — because no provider measures capability at all. The paid ed-tech market therefore sells a product whose effectiveness is, by construction, unmeasured.

2.4 Study of existing computerised systems

Category	Example / description	Why it does not resolve the problem
Video course platforms	Udemy, Coursera, YouTube courses	Linear sequence, no learner model, completion measured rather than capability. Trains recognition exclusively.
Interactive coding platforms	Codecademy, freeCodeCamp, Scrimba	Improve on video by requiring code entry, but exercises are fixed for all learners, heavily scaffolded throughout, and progress is still sequence position rather than a mastery estimate.
Practice and assessment sites	LeetCode, HackerRank, Exercism	Provide execution and grading, which is genuinely valuable, but assume the learner already knows what to practise. No curriculum, no prerequisite model, no adaptive difficulty.
Spaced-repetition tools	Anki, Quizlet	Correctly implement retention scheduling, but operate on facts rather than skills, and cannot assess whether a learner can*construct* anything. Card authoring is fully manual.
General-purpose AI assistants	ChatGPT, Claude, GitHub Copilot	Answer on request, which produces dependency. No persistent model of the learner, no curriculum, no notion of appropriate difficulty, and a commercial incentive to maximise interaction rather than to end it.
AI tutoring products	Khanmigo, Duolingo Max, and similar	Introduce a learner-facing conversational layer over an existing linear curriculum. Retain the engagement objective, and therefore retain the incentive to be maximally helpful at each turn.

Gap identified: no existing system maintains a **separate model of reading and writing ability per skill**, updates it **only from executed evidence**, selects the next step from a **prerequisite graph** rather than a fixed sequence, and **deliberately withdraws its own assistance on a schedule**. Each of these mechanisms is

individually established in the learning-sciences literature. None of them is commercially implemented together, because doing so reduces engagement metrics — which is a business-model constraint, not a technical one, and is therefore an opening.

---

3. Description of the Proposed System

The proposed system is a **web-based, online application** accessible through a standard browser with no installation required, comprising a learner-facing session interface, an embedded code editor, a sandboxed execution service, and a set of backend services maintaining a per-learner model of skill mastery.

3.1 Governing design principle

> **The system optimises for the learner's independent capability over time, not for the learner's satisfaction within any individual interaction.**

Every design decision in this document derives from that statement. Where the two objectives conflict — and they frequently do — independent capability takes precedence. The clearest expression of this is that the system will refuse to supply solution code, and treats every occasion on which it was forced to supply an answer as a recorded system failure rather than a service rendered.

3.2 Positioning and scope decisions

Single domain in version one. The system implements one skill graph — JavaScript through to React — comprising approximately sixty nodes, hand-authored. Breadth is deferred deliberately; a shallow graph across many domains would not demonstrate the mechanism and would not be defensible.

Text, code and exercises only. No video content. Video is the medium whose failure mode this project exists to address, and reintroducing it would place the product in a category with many well-funded incumbents.

The learner is the unit of value, not the content. The system's asset is the accumulated model of what each learner can do and the evidence supporting it. Course content is a commodity; a calibrated learner model is not.

Human-authored curriculum, machine-generated exercises. The skill graph and its prerequisite structure are authored by hand because they are the spine of the system and must be correct. Individual exercises are generated per learner and per difficulty rung, and are executed against their own tests before ever being served.

3.3 Distinguishing characteristics

Three mechanisms differentiate this system, and each is enforced structurally in code rather than stated as an intention.

1. Separate reading and writing mastery per skill. Every skill node carries two independent scores for each learner: the ability to comprehend code that uses the skill, and the ability to produce it unaided. These diverge substantially in practice — the author's own measured profile was 10/11 and 0/4 respectively — and a single blended score would have produced an entirely incorrect next-step recommendation. This split is the central design decision of the system.

2. Evidence-only mastery updates. A mastery score moves when the learner submits code that is executed and passes its tests, when a prediction about program behaviour is correct, or when an exercise is solved. It never moves because content was marked as read, watched or completed. Evidence is weighted by exercise difficulty, by the number of hints consumed, and by time taken.

3. Scheduled withdrawal of scaffolding. Assistance reduces over time on a planned schedule rather than in reaction to learner requests. Early sessions provide worked examples; intermediate sessions provide partially completed files; later sessions provide a specification and an empty file. The fade is a configured property of the learner's plan, not an emergent behaviour.

3.4 The learner model

The learner model is the substantive engineering contribution of the project and the component on which every other module depends.

Skill representation. Skills are stored as a directed acyclic graph with explicit prerequisite relationships, rather than as an ordered list. Selecting a next step becomes a query over this graph.

Mastery representation. For each learner and each skill, the system stores a reading score, a writing score, the timestamp of the most recent supporting evidence, attempt and success counts, and a per-skill decay constant.

Mastery decay. Effective mastery is computed as the stored score attenuated exponentially by time elapsed since the last supporting evidence, with the decay constant varying by skill type — syntactic knowledge decays rapidly, conceptual knowledge slowly. When effective mastery falls below a threshold, the skill re-enters the review queue. This addresses deficiency 6 in §2.2 as a property of the data model rather than as a feature bolted alongside it.

Next-step selection. Selection is deterministic and computed by the system, not delegated to a language model. Candidate skills are those whose prerequisites are sufficiently mastered, together with those due for review. Candidates are scored on prerequisite readiness, relevance to the learner's stated goal, urgency derived from decay, and recency of coverage. The highest-scoring candidate is chosen and the exercise difficulty rung is set from the writing score. A language model then generates the *content* of that step. Preserving this boundary — the system decides *which* step, the model produces *what is in it* — is what makes the system explainable and inexpensive to operate.

3.5 The exercise ladder

Difficulty is a five-position control rather than a binary. The learner model selects the position.

Rung	Form	Cognitive demand	Selected when
1	**Predict** — given code, state the output	Recognition	Skill newly introduced
2	**Diagnose** — given broken code, locate the fault	Analysis	Reading score rising, writing score near zero
3	**Complete** — working file with one to three omissions	Guided production	First production attempt
4	**Extend** — working feature, add the next behaviour	Production with scaffolding	Writing score intermediate

Rung	Form	Cognitive demand	Selected when
5	**Build** — specification, empty file, test suite	Full recall and construction	Writing score high

A blank page is presented only at rung five. This rule exists because the blank page is the observed point of session abandonment, and abandonment is the failure mode the system is designed to prevent.

3.6 The Socratic response engine

The refusal to supply solutions is enforced by three independent mechanisms, because a constraint expressed only in a model prompt is not a constraint.

Server-side hint ladder. Assistance is emitted at escalating levels of specificity — indicating the relevant concept, then the relevant location, then naming the mechanism, then presenting an analogous worked example, and only finally the answer itself. The currently permitted level is held server-side and advances only on explicit learner request. The language model receives one level at a time and never the full ladder.

Output filtering. Generated responses are inspected before transmission for solution-shaped code matching the current exercise target, and are blocked if found.

Bottoming-out as telemetry. Every occasion on which the ladder reaches its final level is recorded as an indication that the step presented was too large, and is fed back into difficulty calibration.

3.7 Users of the system

User	Role	Primary interaction
Learner	The individual acquiring skills	Sessions, exercises, diagnostic, progress view
Curriculum author	Authors and maintains skill graphs and template exercises	Graph editor, exercise bank management
Administrator	Operates the platform	User management, model calibration monitoring, cost and

User	Role	Primary interaction
		usage oversight

Version one serves the learner role fully; the remaining two are operated by the project team through administrative interfaces.

---

4. Description and Identification of the Functional Modules

#	Module	Function	Primary users
M1	Authentication & Account Management	Registration, login, session management, learner profile and goal configuration	Learner, Admin
M2	Skill Graph Management	Storage and traversal of skills, prerequisite relationships, difficulty metadata; authoring interface	Curriculum author
M3	Diagnostic Assessment	Adaptive placement assessment establishing initial position within the graph	Learner
M4	Learner Mastery Model	Per-skill reading and writing scores, evidence-weighted updates, time-based decay, review queue maintenance	System
M5	Next-Step Recommender	Deterministic candidate selection and scoring; rung selection from mastery state	System
M6	Exercise Generation & Validation	Language-model generation of exercise content per skill and rung; self-verification by executing the reference solution against generated tests; fallback to the authored template bank	System
M7	Sandboxed Execution & Grading	Isolated, network-disabled, resource-capped execution of submitted code against tests; verdict production	System

#	Module	Function	Primary users
M8	Socratic Response Engine	Escalating hint ladder with server-side level control, output filtering, struggle detection	Learner
M9	Session Orchestration	Session lifecycle, step sequencing, progress persistence, resumption after interruption	Learner
M10	Spaced Repetition Scheduler	Review scheduling derived from decay; queue prioritisation	System
M11	Progress & Mastery Visualisation	Graph view coloured by effective mastery; honest representation of weak areas; reading versus writing comparison	Learner
M12	Scaffolding Fade Controller	Configurable withdrawal schedule; enforcement of the permitted assistance level per session	System
M13	Calibration & Analytics	Comparison of predicted mastery against held-out challenge outcomes; hint-consumption trends; exercise effectiveness data	Admin

4.1 Division of Work

Member	Roll No.	Responsibility
Ashwani Kumar Puri	**04914004425**	Backend engineering — architecture, database design, the learner model and mastery mathematics (M4), next-step recommender (M5), exercise generation and validation pipeline (M6), sandbox integration (M7), Socratic engine (M8), API design, deployment
Aman Dogra	**01614004425**	Frontend implementation — session interface, embedded editor integration, progress visualisation
Aditi Tanwar	**01814004425**	UI/UX design, curriculum authoring support, frontend implementation

> **Action required:** team composition to be confirmed and entered before submission.

Presentations and viva-voce are individual. Every member is expected to be able to explain the complete system architecture, not solely their own contribution.

---

5. Tools / Platforms

5.1 Hardware Specification

Development machine *(confirm actual figures before submission)*

Component	Specification
Processor	Multi-core x86-64, 8 threads or greater
Memory	16 GB minimum
Storage	512 GB SSD
Network	Broadband internet connection

Deployment target: cloud-hosted; managed application platform for the web tier, managed PostgreSQL, and a managed code-execution service for the sandbox.

5.2 Software Specification

Layer	Technology	Justification
Frontend	Next.js (React), TypeScript	Server-side rendering for public concept pages; component model suited to an editor-centric interface
Code editor	Monaco Editor	The editor component underlying Visual Studio Code; provides syntax awareness in the browser
Backend	Node.js, Express, TypeScript	Shares a runtime with the language being taught, simplifying the execution path

Layer	Technology	Justification
Database	PostgreSQL with pgvector extension	Relational modelling of the skill graph and mastery records, with vector storage for concept retrieval in the same system
Cache & queue	Redis with BullMQ	Session state, review queue, asynchronous execution and generation jobs
Code execution	Managed sandbox service	Security-critical component; a managed runner is used rather than a bespoke implementation
Language models	Two-tier: a small model for hints and grading, a larger model for exercise generation and explanation	Cost control — the majority of interactions are hints, which do not require the larger model
Version control	Git, GitHub	Source control and documentation artefacts held alongside code

6. Methodology

6.1 SDLC Model to be Used

Incremental and iterative development, organised into defined phases, with a preliminary manual validation phase preceding any implementation.

6.2 Justification for the Selection of the Model

The central hypothesis of this project — that adaptive laddered exercises with scheduled scaffolding withdrawal produce better retained capability than linear course consumption — is testable without software. Phase 0 therefore operates the entire loop manually, on paper, with a real learner, before any implementation begins.

This ordering is deliberate and is defended as follows: if the hypothesis fails on paper it will fail in software, and the failure will have cost two weeks rather than several months. Building the machinery first would represent significant engineering investment in an unvalidated premise.

Subsequent phases are incremental because the learner model requires empirical calibration. Each increment produces a working system whose predictions can be compared against observed learner performance, and the calibration improves with each cycle. A waterfall model would defer all such feedback to a single point after implementation, which is unsuitable for a system whose correctness is statistical rather than functional.

6.3 Testing Approach

Unit testing of the mastery-update and decay mathematics against constructed evidence sequences; integration testing of the generation-and-validation pipeline including deliberate injection of invalid generated exercises; security testing of the sandbox against resource exhaustion, network egress and filesystem escape; system testing of complete learner journeys; and **calibration testing**, in which predicted mastery is compared against outcomes on held-out challenge exercises. The calibration error rate is treated as a first-class quality metric of the system rather than as a diagnostic.

6.4 Project Schedule

Aligned to the milestone schedule in the MCA-269 Guidelines. *(Confirm exact dates with the guide.)*

Guideline milestone	Deliverable
Week 2	Topic finalisation and synopsis submission
Week 3–4 · Review 1	Problem analysis, literature survey, Software Requirements Specification
Week 6–7 · Review 2	System design (DFD, ER, UML) and partial implementation

Guideline milestone	Deliverable
Week 9–10 · Review 3	Implementation, testing and draft report
Week 11	Final report submission and internal viva
Week 12	External viva-voce and live demonstration

6.5 Documentation Artefacts

Beyond the prescribed report, the following are maintained in the repository alongside the code, in the form a professional software team would keep them: Software Requirements Specification; system design document containing ER, DFD and UML models; architecture decision records; a prioritised product backlog with acceptance criteria; test plan and test report; and a deployment runbook.

---

7. Commercial Viability

The system is developed as a product rather than as an academic exercise. This section records the commercial reasoning.

7.1 The market problem being sold against

Purchasers of technical training — individuals, employers funding upskilling, and educational institutions — currently have no means of verifying that training produced capability. Providers report completion; buyers require competence. This system measures capability directly and produces evidence for it, which is a materially different product from the one presently on sale.

7.2 Revenue model

Individual subscription. Monthly access for self-directed learners. This is the entry route and the source of early calibration data.

Institutional licensing. Bootcamps, training providers and colleges purchase seats. The buyer's incentive here aligns with the system's design in a way that the consumer market's does not: an institution accountable for graduate outcomes benefits from a product that measures capability honestly, whereas a consumer may prefer a product that feels comfortable. This is the stronger commercial route and the design does not have to be compromised to serve it.

Employer-facing verified capability profiles. A longer-term route in which the mastery model, backed by executed evidence, functions as a credential. This is credible precisely because mastery is only ever updated from executed evidence, and is not available to any provider that records completion.

7.3 Defensibility

The prompts are not defensible; the curriculum graph is moderately so; the **accumulated attempt data** is strongly so. Records of which exercise sequences produced measurable capability gains, across many learners, constitute a dataset that improves the system's difficulty calibration and cannot be obtained without having operated the system. A well-funded competitor can replicate the interface in weeks and cannot replicate that dataset at all.

7.4 The structural advantage

Incumbent products are constrained by engagement metrics. A product measured on time-in-application cannot adopt a design whose explicit goal is that the user finishes sooner and returns less. This system treats **time-in-application as an anti-metric** — a reduction in it, at constant capability gain, is a product improvement. That position is available only to an entrant, and it is the commercial reason this project is viable despite the presence of far larger competitors.

7.5 Validation gate

The project carries an explicit commercial validation gate: a learner who is not the author must complete a multi-session project using the system. Until that gate is passed, the system is a tool built for one person, and this is recorded as a risk rather than obscured.

---

8. Future Scope

The complete web application is delivered within the current semester. The following extend the system beyond that delivery.

1. **Additional skill domains** — Node.js, PostgreSQL, system design — added without modification to the engine, demonstrating that the architecture is domain-independent.
2. **Spoken explanation assessment.** The learner explains a concept aloud and is assessed on the explanation. Explanation quality is among the strongest available indicators of genuine understanding.
3. **Editor integration.** Observation of the learner within their own development environment, with intervention only at points of genuine difficulty.
4. **Community-authored skill graphs**, enabling coverage to grow beyond what the team can author directly.
5. **Cohort mode** — shared paths with light accountability, for institutional deployments.
6. **Effectiveness dataset exploitation** — systematic analysis of which exercise sequences produce measurable capability gains, feeding back into generation.
7. **Portable learner model**, allowing a mastery profile to be exported and recognised across tools.

Appendix — Explicitly Out of Scope

Recorded as decisions rather than omissions. Any addition to scope must displace something already within it.

- Video content of any kind
- Multiple skill domains in version one
- Native mobile applications
- Social features, leaderboards and streak-based gamification — extrinsic motivators displace intrinsic motivation, and streak mechanics penalise precisely the learners this system is designed to serve
- Certification. The mastery graph is the credential
- Payment processing until the institutional phase
- Training of custom language models

- A bespoke code-execution sandbox in version one